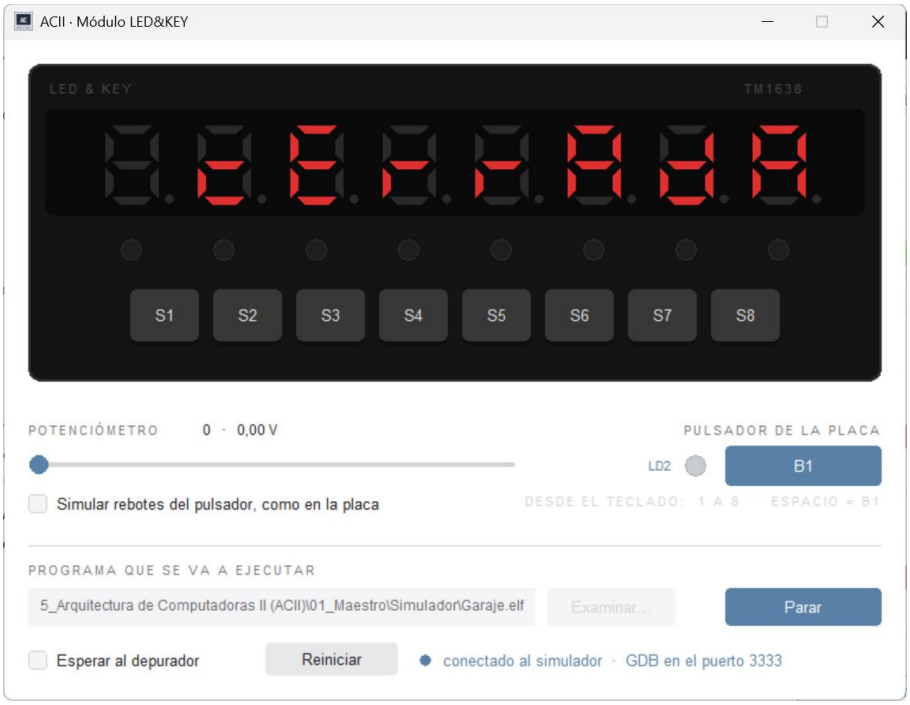
El simulador de la placa

Esta carpeta trae todo lo necesario para **ejecutar y depurar las prácticas sin la placa**, en tu propio ordenador. El mismo binario que grabarías en el microcontrolador, el mismo entorno y el mismo depurador.

Depurar como se ha contado hasta aquí necesita una placa conectada. Hay una forma de no necesitarla: **un emulador** que ejecuta el mismo binario que grabarías en el microcontrolador, con el mismo depurador y los mismos puntos de ruptura. Sirve para trabajar en casa entre sesiones de laboratorio, o cuando alguien está usando la placa.

El emulador se llama **Renode**, es libre, y ejecuta el `.elf` que produce STM32CubeIDE sin cambiarle nada: no hay una versión del programa «para simulador» y otra «para placa». Por delante hay un **panel** que dibuja el módulo LED&KEY —los ocho dígitos, los ocho LED y los ocho pulsadores—, el pulsador azul de la placa y su LED verde, para que haya algo que mirar y algo que pulsar.

El panel es lo único que hay que abrir. Él arranca el emulador, le prepara lo que necesita y se conecta solo; no hace falta consola ni escribir ninguna orden. Y deja siempre listo el servidor al que se conecta el depurador, así que no hay dos modos de arranque ni nada que recordar activar.



EL PANEL CON LA PRÁCTICA 2 EN MARCHA. JUNTO A B1 ESTÁ LD2, EL LED VERDE DE LA PLACA. LAS OCHO TECLAS Y B1 TAMBIÉN RESPONDEN AL TECLADO: 1 A 8, Y LA BARRA ESPACIADORA

De la placa está simulado lo que usan las prácticas:

QUÉ	CÓMO
El microcontrolador	`STM32F401RE` a **84 MHz**, el mismo reloj que configuran los proyectos. `HAL_Delay(1000)` tarda 1000 ms de tiempo simulado
Los puertos GPIO	Entradas y salidas con sus resistencias de pull-up, para que un pulsador sin pulsar se lea como en la placa
El módulo LED&KEY	Los ocho dígitos, los ocho LED y los ocho pulsadores, por los tres pines de siempre
El pulsador azul	En `PC13`, con la misma lógica invertida que en la placa
El LED verde, `LD2`	En `PA5`. Es el que hace parpadear la Práctica 1
El potenciómetro	Como entrada del conversor A/D, de 0 a 4095

QUÉ	CÓMO
Los temporizadores	`TIM2` y `SysTick`, con sus interrupciones
Las interrupciones de pin	`EXTI`: `B1` y las teclas avisan al programa en vez de tener que preguntarles. Es lo que se usa en la Práctica 4

Lo que hay que instalar

El simulador funciona en **Windows, Linux y macOS**. Lo que sigue cuenta primero el camino de Windows, que es el del laboratorio; si trabajas en Linux o en Mac, salta al apartado que lleva su nombre, unas páginas más abajo.

QUÉ	DE DÓNDE	NOTA
Renode, el emulador	En Windows, **ya viene dentro**	En Linux y macOS se instala aparte, y tiene que ser la **1.16.1**
STM32CubeIDE	Ya lo tienes	Es el que compila y el que depura
Python 3 con Tkinter	Solo en Linux y macOS	En Windows no hace falta: allí el panel viene compilado

En Windows no hay que instalar nada aparte: el simulador trae su propio Renode, la versión 1.16.1, que es con la que está probado todo. Da igual que tengas otro instalado; el panel usa el suyo.

> **La versión de Renode importa.** Con otra distinta de la 1.16.1 el emulador puede quedarse parado al arrancar, sin más explicación que «no ha abierto el puerto a tiempo». Por eso en Windows va dentro, y en Linux y macOS el panel avisa si la que encuentra es otra.

Dos formas de tener el simulador

Las dos traen lo mismo y funcionan igual; es cuestión de gusto:

CÓMO	QUÉ HACE
El instalador, `ACII_Simulador_v<versión>.exe`	Solo Windows. Lo instala para tu usuario, sin pedir permisos de administrador, y deja accesos directos al panel y a esta guía en el menú Inicio. Se quita desde «Aplicaciones» como cualquier otro programa
La carpeta comprimida, `ACII_Simulador.zip` o `ACII_Simulador.tar.gz`	Se descomprime donde quieras y se abre el panel. No instala nada, así que vale para llevarlo en un pendrive. El `.tar.gz` es el que hay que coger en Linux y en Mac, porque conserva los permisos de ejecución que el `.zip` pierde

> **El instalador y el ZIP pesan unos 100 MB, y el `.tar.gz` apenas 2.** La diferencia es Renode: en Windows va dentro y en Linux y Mac no, porque allí se instala el del propio sistema.

En Linux y en macOS

Es el mismo simulador: la misma placa, los mismos periféricos y el mismo panel, con el mismo aspecto. Lo único que cambia es que allí el panel está escrito en Python en vez de compilado, y por eso hace falta un Python que traiga Tkinter.

Primero, Renode, y la versión 1.16.1, no la última. Está en `github.com/renode/renode/releases/tag/v1.16.1`. En Linux baja el `.deb` e instálalo:

RENODE EN DEBIAN O UBUNTU

```
sudo apt install ./renode_1.16.1_amd64.deb
```

En Mac baja `renode-1.16.1-dotnet.osx-arm64-portable.dmg` si tu Mac es de Apple Silicon —M1 o posterior—, ábrelo y arrastra `Renode.app` a la carpeta Aplicaciones.

Después, Python. Y aquí hay una trampa que conviene conocer:

SISTEMA	QUÉ HACER
Linux	Tkinter va en un paquete aparte y no suele estar puesto: `sudo apt install python3-tk` en Debian o Ubuntu, `python3-tkinter` en Fedora, `tk` en Arch
macOS	El `python3` que trae el sistema lleva **Tk 8.5**, que Apple dejó de mantener y da problemas de dibujo. Hace falta otro: `brew install python-tk`, o el instalador de `python.org`

Y ya, el simulador. Descomprime el `.tar.gz` donde quieras y ejecuta el instalador que trae dentro, que no pide contraseña ni toca el sistema:

LOS DOS ÚNICOS COMANDOS

```
tar xzf ACII_Simulador.tar.gz
./ACII_Simulador/Panel_py/instalar.sh
```

Ese guion comprueba que todo esté en su sitio y te dice qué falta si falta algo. Cuando termina, deja el simulador a mano:

	LINUX	MACOS
Cómo se abre	Aparece como **«Simulador ACII»** en el menú de aplicaciones	Deja un **«Simulador ACII.command»** en el escritorio, se abre con doble clic
Tipografías	Las copia a `~/.local/share/fonts`	Las copia a `~/Library/Fonts`

También se puede lanzar a mano, sin instalar nada, con `ACII_Simulador/Panel_py/simulador-acii.sh`.

> **Descomprime la carpeta donde vaya a quedarse.** El lanzador que crea el instalador apunta a esa carpeta, no copia nada: si luego la mueves o la borras, deja de funcionar. Si la cambias de sitio, vuelve a ejecutar `instalar.sh` y se arregla.
>
> **Si el panel se ve con otra tipografía**, es que no encontró las suyas. No es grave — funciona igual— pero se arregla ejecutando el instalador, que las deja puestas para toda la sesión de escritorio.

Dónde poner la carpeta

Esta carpeta, donde quieras. La que sí tiene requisitos es **la de tus proyectos de CubeIDE: su ruta no debe llevar espacios ni tildes**. El emulador no sabe abrir ficheros cuya ruta los tenga, y falla de una forma difícil de reconocer: arranca y se queda parado sin decir nada.

RUTA	¿VALE?
`C:\ACII\workspace`	Sí
`D:\Estudios\ACII_2026`	Sí
`/home/juan/acii`	Sí
`C:\Mis Prácticas`	No: lleva espacio y tilde
`C:\Users\Juan\OneDrive\Mis documentos\ACII`	No: lleva espacios
`/Users/juan/Mis prácticas`	No: lleva espacio y tilde

El panel se encarga del resto: copia lo que el emulador tiene que leer —la descripción de la placa, los cuatro modelos y tu `.elf`— a una ruta limpia, y trabaja desde ahí. En Windows es `C:\ACII_sim`; en Linux y en Mac, `~/.acii_sim`. Esa carpeta la crea él; no hay que preparar nada.

> **Este problema no es del simulador.** `make`, CubeMX y el propio depurador se atragantan también con las tildes en la ruta. Es la misma razón por la que el enunciado de la P1 pide guardar los proyectos en una carpeta corta y sin acentos.

Qué hay en esta carpeta

Si lo instalaste con el instalador en vez de descomprimir la carpeta, esto mismo está en `%LOCALAPPDATA%\Programs\Simulador ACII`, y al panel llegas por el menú Inicio.

FICHERO O CARPETA	QUÉ ES
`Panel\PanelACII.exe`	**Lo único que hay que abrir.** El módulo LED&KEY, el pulsador de la placa, y el botón que arranca el emulador
`plataforma/`	La descripción de la placa y los cuatro periféricos que el emulador no trae de serie. No hay que tocar nada de aquí
`ejemplo/Garaje.elf`	Un programa ya compilado, para probar que todo funciona antes de compilar nada tuyo
`Panel/fuentes/`	El código del panel, en Delphi, por si quieres mirarlo
`GUIA DEL SIMULADOR.pdf`	Esto que estás leyendo

Lo primero: comprobar que funciona

Antes de configurar nada en CubeIDE, conviene ver el simulador andando. Abre `Panel\PanelACII.exe`: te propone el programa de ejemplo, que ya viene compilado. Dale a **`Arrancar`**.

El display debe enseñar `cErrAdA`. Al pulsar `B1 (azul)`, la puerta empieza a abrirse y la barra de LED avanza. Si eso pasa, el simulador está bien instalado, y lo que falte será la configuración del depurador.

> La primera vez tarda unos segundos más de lo normal en arrancar: el emulador compila los modelos de los periféricos. Las siguientes va directo.

El panel, control por control

La ventana tiene dos mitades. Arriba, la placa: es lo que tu programa escribe y lo que tú pulsas. Abajo, el mando del simulador: qué programa se ejecuta y cómo.

CONTROL	QUÉ ES
Los ocho dígitos y los ocho LED	El módulo LED&KEY, tal como lo escribe tu programa. Si tu programa no escribe, se quedan apagados
`S1` a `S8`	Los ocho pulsadores del módulo, de izquierda a derecha. Se pulsan con el ratón o con las teclas **1** a **8**
`B1`	**El pulsador azul de la placa**, en `PC13`. No es del módulo: va directo al microcontrolador. Con el ratón, o con la **barra espaciadora**
`LD2`	**El LED verde de la placa**, en `PA5`. Tampoco es del módulo, y por eso va al lado de `B1` y no con los rojos. Se enciende cuando tu programa pone ese pin a 1
`Potenciómetro`	La entrada del conversor A/D. La barra va de **0 a 4095**, y al lado se ve también en voltios
`Simular rebotes del pulsador`	Manda una ráfaga de pulsos al pulsar, como hace un pulsador de verdad, en vez de una transición limpia
`Programa que se va a ejecutar`	El `.elf` que se cargará. Se escribe a mano o se elige con `Examinar…`
`Arrancar` / `Parar`	Levanta el emulador con ese programa, o lo mata

CONTROL	QUÉ ES
`Reiniciar`	Las dos cosas seguidas: para el que hay y levanta otro. Es lo que hay que darle después de recompilar
`Esperar al depurador`	Arranca con el procesador congelado antes de la primera instrucción, para poder depurar desde `main`

> **La casilla de rebotes merece la pena.** En la placa, un pulsador no se cierra de golpe: rebota unos milisegundos. Un programa sin antirrebote parecerá correcto con una pulsación limpia y contará varias en el laboratorio. Con la casilla marcada, el simulador te enseña ese problema en casa en vez de escondértelo.

El teclado. Mientras el simulador está en marcha, las teclas **1** a **8** son `S1` a `S8` y la **barra espaciadora** es `B1`; también valen las del teclado numérico. Se mantienen pulsadas mientras mantengas la tecla, así que se puede probar una pulsación larga. Si escribes en el campo del programa, los atajos se desactivan mientras el foco esté ahí —si no, no podrías teclear una ruta, que lleva dígitos y espacios—; para recuperarlos, `Intro` o un clic en la placa.

La línea de estado, abajo a la derecha, dice en qué punto está:

LO QUE PONE	QUÉ SIGNIFICA
`sin conexión`	No hay emulador; el panel está esperando a que le des a `Arrancar`
`arrancando el emulador…`	Renode está levantando la máquina. La primera vez tarda: compila los modelos
`conectado al simulador`	Todo en marcha. Al lado dice el puerto del servidor GDB

LO QUE PONE	QUÉ SIGNIFICA
`esperando al depurador…`	Con la casilla marcada: el procesador está congelado y no ejecutará nada hasta que se conecte CubeIDE
`el simulador ha cerrado la conexión`	El emulador se ha muerto o lo han matado. El módulo se apaga para no enseñar un fotograma viejo

Ejecutar un programa

Sin depurador, solo para ver si el programa hace lo que debe:

1. **Compila** en CubeIDE como siempre, `Project` → `Build Project`. Mira en la consola que acaba con `0 errors`.
2. **Localiza el `.elf`.** Está en la carpeta `Debug` del proyecto y se llama como el proyecto: si tu proyecto es `P3_Reloj`, el fichero es `...\P3_Reloj\Debug\P3_Reloj.elf`. En el Project Explorer aparece bajo `Binaries`.
3. **Abre el panel** —en Windows `Panel\PanelACII.exe`; en Linux y en Mac, el lanzador que dejó el instalador— y elige ese `.elf` con `Examinar…`.
4. **Dale a `Arrancar`.** El panel copia lo que hace falta, lanza el emulador y se conecta solo. Cuando la línea de estado ponga `conectado al simulador`, el programa ya está corriendo.

A partir de ahí el panel es la placa: los dígitos y los LED enseñan lo que escribe el programa, y `S1`..`S8`, `B1` y el potenciómetro le llegan como si los pulsaras. Para terminar, `Parar`.

> **La primera vez tarda más.** El emulador compila al vuelo los cuatro modelos de los periféricos, y eso son unos segundos. Las siguientes va directo. Si pasa un minuto sin arrancar, algo va mal: mira el apartado de problemas.

> **Al recompilar hay que volver a arrancar.** El emulador carga el binario una vez, al empezar. Después de recompilar en CubeIDE, `Reiniciar` —que para el emulador y levanta otro con el binario nuevo—, o seguirás viendo el programa anterior, que es la forma más tonta de perder una tarde.

Con el simulador en marcha, el campo del programa y `Examinar…` quedan cerrados: el que se está ejecutando es el que se cargó al arrancar, y para cambiarlo hay que parar.

Depurar desde STM32CubeIDE

Se puede poner un punto de ruptura en CubeIDE y que se cumpla dentro del emulador, con su código fuente, sus variables y su pila de llamadas. Es el depurador de siempre; lo único que cambia es a qué se conecta.

El emulador **siempre** abre un servidor GDB en el puerto **3333**, estés depurando o no, así que puedes conectarte en cualquier momento. La configuración del lado de CubeIDE se hace **una vez por proyecto**, y te la deja hecha el propio panel.

Paso 1. Arranca el simulador. Elige el `.elf` y dale a `Arrancar`, igual que antes. Déjalo corriendo: el depurador se va a conectar a él.

Si lo que quieres es **parar dentro de `main`, antes de que se ejecute nada**, marca la casilla **`Esperar al depurador`** antes de darle a `Arrancar`. El emulador queda entonces con el procesador congelado en la primera instrucción y no arranca hasta que el depurador se lo diga. El display se queda apagado y el estado pone `esperando al depurador…`: es lo que tiene que pasar.

Paso 2. La configuración de depuración. Al arrancar, el panel mira el proyecto al que pertenece el `.elf`, y si todavía no tiene configuración de depuración para el emulador se la deja hecha. Te lo dice con un aviso. Aparece en CubeIDE en `Run` → `Debug Configurations…`, bajo **`GDB Hardware Debugging`**, con el nombre **`Simulador ACII - <proyecto>`**. Lleva el nombre del proyecto porque ahí se listan las de todos los que tengas abiertos, y si se llamaran igual no se distinguirían.

> **Si el proyecto ya estaba abierto en CubeIDE, refréscalo con F5** —selecciona el proyecto en el Project Explorer y pulsa F5— para que vea el fichero nuevo. Si aun así no aparece, cierra y vuelve a abrir CubeIDE.

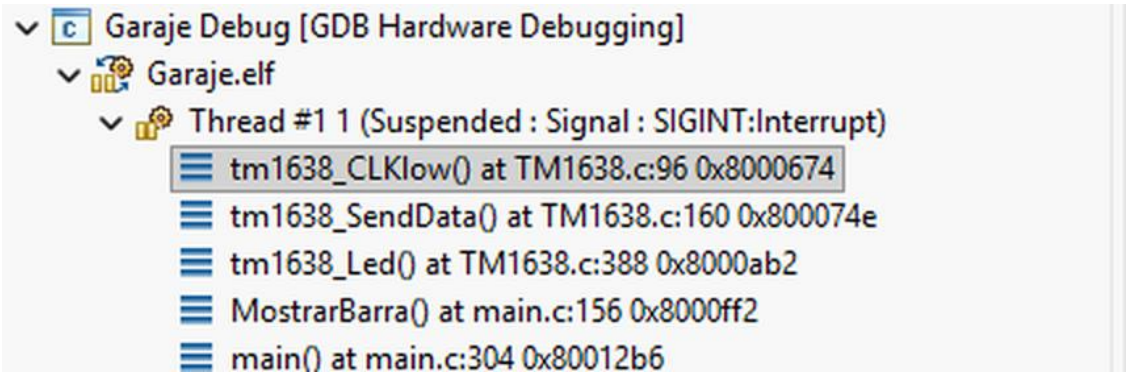
Viene marcada como **favorita**, así que también la tienes en la flecha de al lado del botón de depurar, arriba del todo y por delante del historial.

> **El botón de depurar a secas, sin desplegar, no sirve la primera vez.** Relanza siempre lo último que se lanzó, que será la configuración de la placa. Hay que desplegar la flecha de al lado y elegir la del simulador; a partir de ahí el botón ya repite esa.
>
> **No es la misma que la de la placa.** La de siempre, `STM32 C/C++ Application`, habla con el ST-LINK y sigue donde estaba, para cuando depures en la placa. Son dos configuraciones distintas del mismo proyecto y no se estorban.

Paso 3. Pon los puntos de ruptura y dale a Debug. Al conectarse, el depurador te lleva a la perspectiva de depuración. Lo que ves depende de cómo hayas arrancado:

CÓMO ARRANCASTE	DÓNDE SE DETIENE
Sin `Esperar al depurador`	Donde estuviera el programa, que casi siempre es dentro de la biblioteca del módulo: `Suspended : Signal : SIGINT`. La inicialización ya pasó
Con `Esperar al depurador`	En la primera línea de `main`, con **nada** ejecutado. Desde ahí se puede recorrer `SystemClock_Config` y los `MX_..._Init` paso a paso

A partir de ahí es el depurador de siempre: F5 para entrar, F6 para pasar por encima, F8 para continuar, la vista de variables, la de registros y la pila de llamadas.



EL PROGRAMA DETENIDO DENTRO DE LA BIBLIOTECA DEL MÓDULO, CON LA PILA DE LLAMADAS QUE HA LLEVADO HASTA AHÍ

Para volver a depurar desde el principio, el botón `Reiniciar` del panel. Parar el programa en CubeIDE, o pedirle que lo relance, no sirve: al procesador solo se le puede coger antes de la primera instrucción con un emulador recién arrancado, y eso es lo que hace ese botón. Después, `Debug` otra vez.

Y el ciclo completo mientras trabajas —cambias el código, quieres volver a probarlo— es este:

5. En CubeIDE, `Terminate` para soltar el depurador.
6. `Project` → `Build Project` para recompilar.
7. En el panel, `Reiniciar`.
8. `Debug` otra vez, eligiendo `Simulador ACII - <proyecto>` en la flecha.

Lo que lleva dentro la configuración

No hay que tocarlo: el panel lo escribe. Se cuenta porque es lo que habría que poner a mano en un proyecto que no haya pasado por el panel, y porque aclara por qué el depurador y el emulador se entienden.

En la pestaña **`Main`**, el proyecto y su binario: `Debug\<nombre>.elf`.

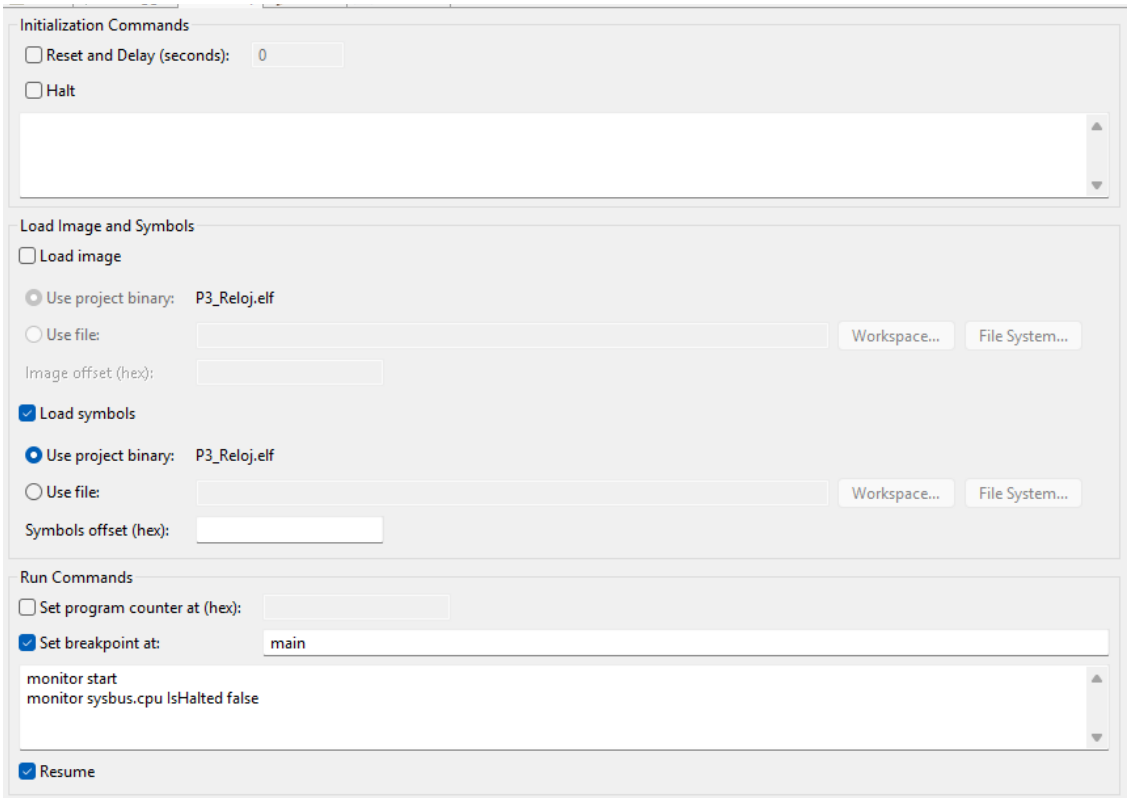
En la pestaña **`Debugger`**, a dónde conectarse:



LA PESTAÑA DEBUGGER: EL DEPURADOR HABLA CON EL EMULADOR POR UN PUERTO, EN LUGAR DE CON LA SONDA

CAMPO	QUÉ PONER
`GDB Command`	`arm-none-eabi-gdb`, el mismo que usa el entorno para la placa
`Use remote target`	**Marcado.** Es lo que habilita el resto
`Debug server`	`Generic TCP/IP`
`Protocol`	`remote`
`Connection`	`localhost:3333`

Y en la pestaña **`Startup`**, que es donde se tuerce si se deja como viene sola:



LA PESTAÑA STARTUP TAL COMO TIENE QUE QUEDAR: DE LAS CINCO CASILLAS, SOLO «LOAD SYMBOLS» Y «RESUME» MARCADAS; ABAJO, EL PUNTO DE RUPTURA EN MAIN Y LAS DOS ÓRDENES QUE ARRANCAN EL EMULADOR

CASILLA	CÓMO	POR QUÉ
`Reset and Delay`	**Desmarcada**	Es una orden de la sonda física. El emulador no la tiene
`Halt`	**Desmarcada**	Lo mismo. Además el procesador ya llega retenido
Órdenes de inicialización	Vacío	No hay nada que inicializar
`Load image`	**Desmarcada**	El programa ya lo ha cargado el emulador
`Load symbols`	**Marcada**	Es lo que da código fuente, variables y pila de llamadas
`Set breakpoint at`	`main`	Con la casilla del panel marcada, este sí se cumple

CASILLA	CÓMO	POR QUÉ
`Run Commands`	**Las dos órdenes de abajo**	Arrancan el tiempo del emulador y sueltan el procesador
`Resume`	Marcada	Que eche a andar al conectarse

En la caja **`Run Commands`**, la de abajo del todo, van estas dos líneas:

```
monitor start
monitor sysbus.cpu IsHalted false
```

La primera arranca el reloj del emulador y la segunda suelta el procesador. Van ahí y no en las órdenes de inicialización **porque esa caja se ejecuta después de poner los puntos de ruptura**: si se arrancara antes, el programa pasaría por `main` sin que hubiera nada que lo detuviera.

> **`continue` no arranca el emulador por sí solo.** El tiempo de la emulación es cosa de Renode, no del programa, y hay que pedírselo con `monitor start`. Sin esas dos líneas, con la casilla marcada, el depurador se conecta y se queda esperando para siempre.
>
> **Si al lanzar sale «Reset command not defined for debug server 'Generic TCP/IP'», es que `Reset and Delay` sigue marcada.** Desmárcala, y desmarca también `Halt` o darás con el mismo mensaje cambiando la primera palabra. Son órdenes que solo entiende una sonda de verdad.

Lo que el simulador no puede enseñar

El tiempo no es el de la placa. El emulador va del orden de dos veces más lento, y ese factor depende del ordenador. Sirve para ver **qué hace** el programa; para saber **cuánto tarda**, la placa.

ESTO SÍ	ESTO NO
Si el programa llega donde debe y con qué valores	Cuánto tarda una rutina
Si la máquina de estados hace las transiciones correctas	Calibrar un bucle de espera
Qué escribe en el display y qué LED enciende	El brillo real del display
Si una tecla se detecta y si el antirrebote funciona	Un contacto sucio o un cable flojo

> **El simulador no sustituye al laboratorio, lo complementa.** Un programa que funciona en el simulador puede fallar en la placa, y eso no es un defecto del simulador: es que el hardware tiene comportamientos que ningún modelo recoge. La entrega se comprueba en la placa.

Si algo no va

LO QUE VES	QUÉ PASA Y QUÉ HACER
«No encuentro Renode»	En Windows, falta la carpeta Renode junto a la del panel: vuelve a instalar, o a descomprimir el ZIP entero. En Linux y Mac, no está instalado
«Otra versión de Renode»	Solo en Linux y Mac. Tienes una distinta de la 1.16.1. Puedes seguir, pero si no arranca, es lo primero que hay que cambiar
«El emulador se ha cerrado nada más arrancar»	El panel enseña lo último que dijo el emulador. Casi siempre es que la ruta del proyecto lleva espacios o tildes

LO QUE VES	QUÉ PASA Y QUÉ HACER
«Ya hay un simulador corriendo»	Quedó uno de antes, de otra ventana del panel. Ciérrala y vuelve a darle a `Arrancar`
«El emulador no ha abierto el puerto a tiempo»	Ha tardado más de dos minutos y medio. Mira el registro: casi siempre es una pieza que falta en la carpeta `plataforma`
El panel arranca pero el display no cambia	El programa no escribe en el módulo. Es un fallo del programa, no del simulador: pon un punto de ruptura donde escribe
El display se queda apagado y no reacciona	Si marcaste `Esperar al depurador`, es lo esperado: no ejecutará nada hasta que conectes CubeIDE
`Simulador ACII - ...` no aparece en CubeIDE	Refresca el proyecto con F5. Si sigue sin salir, cierra y abre CubeIDE
El depurador se conecta y se queda colgado	Faltan las dos líneas de `Run Commands`. Ver el apartado anterior
Al depurar, un punto de ruptura en la inicialización no se cumple	Arrancaste sin `Esperar al depurador`: cuando te conectas ya había pasado. `Parar`, marcar la casilla y `Arrancar`
Las teclas 1..8 no hacen nada	El foco está en el campo del programa. `Intro`, o un clic en la placa

Dónde mirar cuando nada de lo anterior encaja. El emulador deja todo lo que va diciendo en un registro, y el panel deja ahí también el guion con el que lo ha lanzado. La carpeta de trabajo es `C:\ACII_sim` en Windows y `~/.acii_sim` en Linux y en Mac:

FICHERO	QUÉ ES
`<carpeta de trabajo>/renode.log`	Todo lo que ha dicho el emulador, incluido por qué se ha cerrado
`<carpeta de trabajo>/simula.resc`	Las órdenes con las que se ha lanzado: qué placa, qué modelos y qué `.elf`
`<carpeta de trabajo>/*.elf`	La copia del programa que se está ejecutando. Si la fecha no es la de tu última compilación, no le diste a `Reiniciar`